

Engineer's Odyssey™

Correct Answers, Hints & Java Reference Solutions

Use this guide only after an assessment attempt. Revisit the relevant Java concept, then retry the level to earn its badge and Kamil-signed certificate.

MCQ answers and hints

No.	Correct answer	Hint
1	new	Objects are instantiated with the new keyword.
2	public static void main(String[] args)	This is the Java application entry point.
3	int	Use int for a whole number in the normal integer range.
4	==	Use == for primitive equality; use equals() for String content.
5	for	A for loop works well when the index range is known.
6	ArrayList	ArrayList preserves insertion order and allows duplicates.
7	length	Arrays expose the length field; collections use size().
8	NullPointerException	A catch block can handle a thrown exception.
9	return	return exits a method and optionally supplies a result.
10	private	private limits access to the declaring class.
11	inheritance	Inheritance lets a class reuse behavior with extends.
12	Scanner	Scanner is commonly used for console input.
13	Map	Map stores key-value pairs.
14	continue	continue skips to the next loop iteration.
15	O(n)	One pass over n items is linear time.

Java coding & debugging reference

1. Read and print an integer

```
import java.util.Scanner;
public class Main { public static void main(String[] args) { Scanner sc = new
Scanner(System.in); int n = sc.nextInt(); System.out.println(n); } }
```

Hint: Start with public class Main, read an int using Scanner, then print it.

2. Write and return a method result

```
public class Main { static int square(int n) { return n * n; } public static void
main(String[] args) { System.out.println(square(5)); } }
```

Hint: A Java method needs a return type, parameters, and return for a non-void result.

3. Process an array

```
public class Main { static int sum(int[] arr) { int total = 0; for (int value : arr) total +=
value; return total; } }
```

Hint: Use a for-each loop for a simple full-array traversal.

4. Test a solution in main

```
public class Main { static boolean isEven(int n) { return n % 2 == 0; } public static void
main(String[] args) { System.out.println(isEven(8)); } }
```

Hint: Call your method from main with a known input.

5. Use an ArrayList

```
import java.util.ArrayList; public class Main { public static void main(String[] args) {
ArrayList<Integer> values = new ArrayList<>(); values.add(10);
System.out.println(values.get(0)); } }
```

Hint: Import ArrayList, use add(), then retrieve with get(index).

6. Fix a missing semicolon

```
System.out.println("Ready");
```

Hint: Java statements usually end with a semicolon.

7. Compare Strings correctly

```
if (name.equals("Kapil")) { System.out.println("Match"); }
```

Hint: Use equals() for String content comparison, not ==.

8. Fix an array boundary

```
for (int i = 0; i < arr.length; i++) { System.out.println(arr[i]); }
```

Hint: The final valid array index is length - 1, so use < arr.length.

Java reference - continued

9. Guard against null

```
if (value != null) { System.out.println(value.length()); }
```

Hint: Check references before invoking a method on them.

10. Handle invalid input

```
try { int n = Integer.parseInt(text); } catch (NumberFormatException e) {  
System.out.println("Invalid number"); }
```

Hint: Wrap risky parsing in try/catch and catch the specific exception.

Before you retry

Read the compiler message carefully, test the smallest valid input, check array bounds, and confirm whether values can be null. Then submit a fresh Java-only solution. A pass requires 90% or more in both the MCQ and coding/debugging sections.